```
  ...
}
```

❶ Creates a behavior with a monitor probe

Once the probe and the behavior under test are defined, you can create a monitor with them and spawn it. This monitor actor has the probe that intercepts any message sent to the behavior, and the behavior under test. Every time you send a message to the monitor, the probe intercepts the message and forwards it to the behavior. This way, you can prove that the actor under test receives the message by testing its probe.

## 4.3.2 Fishing for messages

Let’s say you have a counter that can pause for a few seconds and then resume automatically. During the pause, the counter can’t be incremented, as shown in this listing.

**Listing 4.3 Fishing for messages in a counter**

```
object CounterTimer {

  sealed trait Command
  final case object Increase extends Command
  final case class Pause(seconds: Int) extends Command
  private[fishing] final case object Resume extends Command

  def apply(): Behavior[Command] =
    resume(0)

  def resume(count: Int): Behavior[Command] =
    Behaviors.receive { (context, message) =>
      Behaviors.withTimers { timers =>
        message match {
          ...
          case Pause(t) =>
            timers.startSingleTimer(Resume, t.second)     ❶
            pause(count)
  }}}

  def pause(count: Int): Behavior[Command] = {
    Behaviors.receive { (context, message) =>
      message match {
        ...
```